ClientLens — Complete Solution

> The whole project, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck. All of this runs against the example servers (the model calls are mocked in the tests).

The final layout

```
<repo folder>/
├── servers/
│   ├── orders_server.py
│   └── tickets_server.py
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── orders_agent.py
│   │   ├── supervisor.py
│   │   ├── tickets_agent.py
│   │   └── writer_agent.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── graph.py
│   ├── mcp_client.py
│   └── state.py
├── tests/
│   ├── integration/
│   │   └── test_live_crew.py
│   └── unit/
│       ├── test_config.py
│       └── test_supervisor.py
├── .env.example
├── .gitignore
├── LICENSE
```

```
├── main.py
└── requirements.txt
```

Setup files

`requirements.txt`: `mcp`, `openai`, `langgraph`, `python-dotenv`, `pydantic`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

The two MCP servers

```
# servers/orders_server.py
# A minimal orders MCP server. A real deployment would point this at the real
# orders service; this is a dict on purpose, for repeatable teaching.
import sys
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("orders-server")
ORDERS = {
    "c-42": ["#1001 Laptop $1200 - delivered",
             "#1042 USB-C Dock $150 - shipped"],
}

@mcp.tool()
def get_orders(customer_id):
    """Get a customer's recent orders by their customer id."""
    orders = ORDERS.get(customer_id)
    if not orders:
        return "No orders for this customer."
    return "\n".join(orders)

if __name__ == "__main__":
    print("orders server starting", file=sys.stderr)
    mcp.run(transport="stdio")
```

```
# servers/tickets_server.py
# A minimal support-tickets MCP server (mocked).
import sys
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("tickets-server")
```

```
TICKETS = {
    "c-42": ["T-77 OPEN: USB-C dock not charging",
             "T-50 CLOSED: refund issued for late delivery"],
}

@mcp.tool()
def get_tickets(customer_id):
    """Get a customer's support tickets by their customer id."""
    tickets = TICKETS.get(customer_id)
    if not tickets:
        return "No tickets for this customer."
    return "\n".join(tickets)

if __name__ == "__main__":
    print("tickets server starting", file=sys.stderr)
    mcp.run(transport="stdio")
```

Settings, errors, logging

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str
    connect_timeout: float

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value
```

```
def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        connect_timeout=float(os.getenv("MCP_CONNECT_TIMEOUT", "10")),
    )
```

```
# src/utils/logger.py
import logging
import os
import sys

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler(sys.stderr)
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from openai import OpenAI

def create_client(config):
    return OpenAI(api_key=config.openai_api_key)
```

The shared MCP specialist loop (with degradation)

```
# src/mcp_client.py
# The shared specialist loop: connect to one server, discover its tools, run
# a ReAct loop, return (findings, unavailable). The connection is opened and
# closed inside this one async call, so it runs cleanly as a graph node. The
# try/except is the degradation path: a dead server -> ("", True).
import asyncio
import json

from mcp import ClientSession, StdioServerParameters
```

```
from mcp.client.stdio import stdio_client

from src.utils.logger import get_logger

MODEL = "gpt-4o-mini"

def _openai_tools(tools):
    schemas = []
    for tool in tools:
        schemas.append({
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description or "",
                "parameters": tool.input_schema,
            },
        })
    return schemas

async def run_specialist(client, command, args, system, task,
                         connect_timeout=10.0):
    logger = get_logger(__name__)
    params = StdioServerParameters(command=command, args=args)
    try:
        async with stdio_client(params) as (read, write):
            async with ClientSession(read, write) as session:
                await asyncio.wait_for(
                    session.initialize(), timeout=connect_timeout
                )
                listed = await session.list_tools()
                tools = _openai_tools(listed.tools)
                messages = [
                    {"role": "system", "content": system},
                    {"role": "user", "content": task},
                ]
                for _ in range(5):
                    reply = client.chat.completions.create(
                        model=MODEL, messages=messages, tools=tools
                    ).choices[0].message
                    if not reply.tool_calls:
                        return reply.content or "", False
                    messages.append({
                        "role": "assistant", "content": reply.content,
                        "tool_calls": _calls_as_dicts(reply.tool_calls),
                    })
                    for call in reply.tool_calls:
                        arguments = json.loads(call.function.arguments)
```

```
                        result = await session.call_tool(
                            call.function.name, arguments
                        )
                        messages.append({
                            "role": "tool", "tool_call_id": call.id,
                            "content": result.content[0].text,
                        })
                return "(no answer)", False
    except Exception as e:
        logger.warning(f"specialist for {args} unavailable: {e}")
        return "", True

def _calls_as_dicts(tool_calls):
    out = []
    for call in tool_calls:
        out.append({
            "id": call.id, "type": "function",
            "function": {"name": call.function.name,
                         "arguments": call.function.arguments},
        })
    return out
```

The two specialists

```
# src/agents/orders_agent.py
from src.mcp_client import run_specialist

ORDERS_SYSTEM = ("You are the Orders specialist. Use the orders tool to pull "
                 "the customer's recent orders. Report only what you found.")

async def run_orders_agent(client, task, connect_timeout=10.0):
    return await run_specialist(
        client, "python", ["servers/orders_server.py"],
        ORDERS_SYSTEM, task, connect_timeout,
    )
```

```
# src/agents/tickets_agent.py
from src.mcp_client import run_specialist

TICKETS_SYSTEM = ("You are the Tickets specialist. Use the tickets tool to "
                  "pull the customer's support tickets. Report only what you "
                  "found.")

async def run_tickets_agent(client, task, args=None, connect_timeout=10.0):
```

```
    if args is None:
        args = ["servers/tickets_server.py"]
    return await run_specialist(
        client, "python", args, TICKETS_SYSTEM, task, connect_timeout,
    )
```

The Supervisor

```
# src/agents/supervisor.py
from langgraph.types import Command

from src.utils.logger import get_logger

def needs_orders(task):
    lowered = task.lower()
    return "order" in lowered or "360" in lowered or "summary" in lowered

def needs_tickets(task):
    lowered = task.lower()
    return "ticket" in lowered or "support" in lowered or "360" in lowered

def decide_next(state):
    task = state["task"]
    if needs_orders(task) and state["orders"] == "":
        return "orders"
    if needs_tickets(task) and state["tickets"] == "" \
            and not state["tickets_unavailable"]:
        return "tickets"
    return "writer"

def supervisor(state):
    logger = get_logger(__name__)
    target = decide_next(state)
    log = state["routing_log"] + [target]
    logger.info(f"supervisor -> {target}")
    return Command(goto=target, update={"routing_log": log})
```

The Writer

```
# src/agents/writer_agent.py
from src.utils.logger import get_logger
```

```
MODEL = "gpt-4o-mini"
WRITER_SYSTEM = (
    "You are the Writer. Synthesize the specialists' findings into ONE short "
    "customer-360 summary (title + a few bullets). Weave orders and tickets "
    "together - do not just list them. If a source is marked unavailable, say "
    "so plainly; never pretend it was checked."
)

def run_writer(client, task, orders, tickets, tickets_unavailable):
    logger = get_logger(__name__)
    tickets_section = tickets
    if tickets_unavailable:
        tickets_section = "(the support-tickets service could not be reached)"
    user = (
        f"Task: {task}\n\nOrders:\n{orders or '(none)'}\n\n"
        f"Tickets:\n{tickets_section or '(none)'}\n\nWrite the 360 summary."
    )
    reply = client.chat.completions.create(
        model=MODEL,
        messages=[
            {"role": "system", "content": WRITER_SYSTEM},
            {"role": "user", "content": user},
        ],
    ).choices[0].message
    logger.info("writer produced the 360 summary")
    return reply.content or ""
```

The state and the graph

```
# src/state.py
from typing import TypedDict

class TeamState(TypedDict):
    task: str
    orders: str
    tickets: str
    tickets_unavailable: bool
    report: str
    routing_log: list

def start_state(task):
    return {
        "task": task, "orders": "", "tickets": "",
```

```
        "tickets_unavailable": False, "report": "", "routing_log": [],
    }
```

```
# src/graph.py
# START -> supervisor; specialist nodes are async (own MCP connection) and
# return Command(goto="supervisor", ...). Run with .ainvoke(). tickets_args
# is overridable so the degradation test can point at a dead server.
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command

from src.agents.orders_agent import run_orders_agent
from src.agents.supervisor import supervisor
from src.agents.tickets_agent import run_tickets_agent
from src.agents.writer_agent import run_writer
from src.state import TeamState

def build_graph(client, connect_timeout=10.0, tickets_args=None):
    async def orders_node(state):
        findings, _ = await run_orders_agent(
            client, state["task"], connect_timeout
        )
        return Command(goto="supervisor", update={"orders": findings})

    async def tickets_node(state):
        findings, unavailable = await run_tickets_agent(
            client, state["task"], tickets_args, connect_timeout
        )
        return Command(goto="supervisor", update={
            "tickets": findings, "tickets_unavailable": unavailable,
        })

    def writer_node(state):
        report = run_writer(
            client, state["task"], state["orders"], state["tickets"],
            state["tickets_unavailable"],
        )
        return Command(goto=END, update={"report": report})

    graph = StateGraph(TeamState)
    graph.add_node("supervisor", supervisor)
    graph.add_node("orders", orders_node)
    graph.add_node("tickets", tickets_node)
    graph.add_node("writer", writer_node)
    graph.add_edge(START, "supervisor")
    return graph.compile()
```

main.py

```
# main.py
import asyncio
import sys

from src.config import load_config
from src.exceptions import MissingConfigError
from src.graph import build_graph
from src.models.llm import create_client
from src.state import start_state

TASK = ("Give me a customer-360 for c-42: their orders and support tickets, "
        "as one summary.")

async def run_demo():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)
    client = create_client(config)
    app = build_graph(client, connect_timeout=config.connect_timeout)
    final = await app.ainvoke(start_state(TASK))
    print("Routing:", final["routing_log"])
    print("\n" + final["report"])

def main():
    asyncio.run(run_demo())

if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "CLIENTLENS_TEST_SETTING"
```

```
def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting():
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

```
# tests/unit/test_supervisor.py
# The routing brain, on hand-made states — no model, no servers.
# Run from the repo root: python -m tests.unit.test_supervisor
from src.agents.supervisor import decide_next
from src.state import start_state

T = "Give me a customer-360 for c-42: orders and support tickets."

def test_fresh_routes_to_orders():
    result = "WRONG"
    if decide_next(start_state(T)) == "orders":
        result = "OK"
    print(f"case 1 (fresh -> orders)    -> {result}")

def test_orders_done_routes_to_tickets():
```

```
    state = start_state(T)
    state["orders"] = "found"
    result = "WRONG"
    if decide_next(state) == "tickets":
        result = "OK"
    print(f"case 2 (orders done -> tickets) -> {result}")

def test_both_done_routes_to_writer():
    state = start_state(T)
    state["orders"] = "found"
    state["tickets"] = "found"
    result = "WRONG"
    if decide_next(state) == "writer":
        result = "OK"
    print(f"case 3 (both done -> writer) -> {result}")

def test_tickets_down_skips():
    state = start_state(T)
    state["orders"] = "found"
    state["tickets_unavailable"] = True
    result = "WRONG"
    if decide_next(state) == "writer":
        result = "OK"
    print(f"case 4 (tickets down -> writer) -> {result}")

if __name__ == "__main__":
    test_fresh_routes_to_orders()
    test_orders_done_routes_to_tickets()
    test_both_done_routes_to_writer()
    test_tickets_down_skips()
```

```
# tests/integration/test_live_crew.py
# Launches the real MCP servers and the model. Run from the repo root:
#   python -m tests.integration.test_live_crew
import asyncio

from src.config import load_config
from src.graph import build_graph
from src.models.llm import create_client
from src.state import start_state

TASK = ("Give me a customer-360 for c-42: their orders and support tickets, "
        "as one summary.")

async def test_both_services(client):
    final = await build_graph(client).ainvoke(start_state(TASK))
```

```
    used = final["routing_log"]
    report = final["report"].lower()
    ok = "orders" in used and "tickets" in used and "writer" in used
    if "c-42" not in report and "laptop" not in report:
        ok = False
    result = "WRONG"
    if ok:
        result = "OK"
    print(f"case 1 (both services -> one 360)       -> {result}")

async def test_tickets_down(client):
    app = build_graph(client, tickets_args=["servers/does_not_exist.py"])
    final = await app.ainvoke(start_state(TASK))
    ok = final["tickets_unavailable"] is True
    report = final["report"].lower()
    if "orders" not in final["routing_log"]:
        ok = False
    if "reach" not in report and "unavailable" not in report:
        ok = False
    result = "WRONG"
    if ok:
        result = "OK"
    print(f"case 2 (tickets down -> honest 360)     -> {result}")

async def main():
    client = create_client(load_config())
    await test_both_services(client)
    await test_tickets_down(client)

if __name__ == "__main__":
    asyncio.run(main())
```

Expected unit output (all `OK`):

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (fresh -> orders)    -> OK
case 2 (orders done -> tickets) -> OK
case 3 (both done -> writer) -> OK
case 4 (tickets down -> writer) -> OK
```

Expected integration output (real servers + model):

```
case 1 (both services -> one 360)       -> OK
case 2 (tickets down -> honest 360)     -> OK
```

That's the whole team: a Supervisor that only routes, two specialists each owning their own MCP connection, a Writer that synthesizes one customer-360, and a dead service that degrades the team into an honest partial summary instead of a crash.